EAST WEST UNIVERSITY

CSE110: Object Oriented Programming (Fall 2025)

Project Report

Smart Task Manager

Course Code	: CSE110
Course Title	: Object Oriented Programming
Instructor	: Dr. Md. Hasanul Ferdaus
Section	: 04
Submitted by	: Sadman Showmik Anik (2025-1-60-055)
Date of Submission	: 26-12-2025

Contents

1 Introduction

The **Smart Task Manager** is an interactive console-based Java application designed to help users organize and manage their daily tasks efficiently. The application allows users to create, track, and manage both personal and work-related tasks through a user-friendly menu-driven interface.

1.1 Motivation and Purpose

The motivation behind creating this application was to develop a practical task management tool that demonstrates core Object-Oriented Programming (OOP) concepts while providing real utility. The application helps users stay organized by categorizing tasks, tracking completion status, and persisting data between sessions.

1.2 Main Features

- **Add Tasks**: Create new work or personal tasks with titles and descriptions
- **Mark Completion**: Mark tasks as completed with status tracking
- **View Tasks**: Display all tasks, completed tasks only, or pending tasks only
- **Data Persistence**: Save and load tasks from file for data retention
- **Interactive Menu**: User-friendly console interface with visual formatting

1.3 OOP Concepts Demonstrated

- **Class and Object**: Multiple classes representing different entities (Task, WorkTask, PersonalTask, TaskManager)
- **Encapsulation**: Private fields with public getter/setter methods
- **Inheritance**: WorkTask and PersonalTask extending the abstract Task class
- **Polymorphism**: Overridden methods providing different behaviors in subclasses
- **Exception Handling**: Custom TaskNotFoundException and try-catch blocks

2 System Design

2.1 Overview of the System Structure

The Smart Task Manager follows a hierarchical class structure with clear separation of concerns:

```
TaskManager (Main Application)
    |
    +-- Task (Abstract Base Class)
    |       |
    |       +-- WorkTask (Subclass)
    |       |
    |       +-- PersonalTask (Subclass)
    |
    +-- TaskNotFoundException (Custom Exception)
```

2.2 Program Flow (Start to End)

1. **Startup**: `TaskManager.main()` creates a TaskManager instance
2. **Load Data**: `loadTasksFromFile()` loads any previously saved tasks
3. **Display Welcome**: Shows welcome banner to user
4. **Main Loop**: Displays menu and processes user choices
5. **User Operations**: Add tasks, mark completed, view tasks, etc.
6. **Save & Exit**: Saves tasks to file and exits gracefully

2.3 Class Descriptions

2.3.1 Class: Task

Attribute	Description
Class Name	Task
Parent Class	None (Abstract base class)
Purpose	Serves as the abstract base class for all task types. Defines common attributes and behaviors that all tasks share.

Table 1: Task Class Overview

Key Fields (Encapsulated Variables):

Field	Type	Access	Description
id	int	private	Unique identifier for each task
title	String	private	The title/name of the task
description	String	private	Detailed description of the task
isCompleted	boolean	private	Completion status flag
idCounter	static int	private	Auto-incrementing counter for unique IDs

Table 2: Task Class Fields

Key Methods:

Method	Return Type	Description
getId()	int	Returns the task's unique ID
getTitle()	String	Returns the task title
setTitle(String)	void	Sets a new title
getDescription()	String	Returns the task description
setDescription(String)	void	Sets a new description
isCompleted()	boolean	Returns completion status
setCompleted(boolean)	void	Sets completion status
getDetails()	abstract String	Abstract method for subclass implementation
getTaskType()	abstract String	Abstract method returning task type
toFileString()	String	Converts task to file-saveable format

Table 3: Task Class Methods

2.3.2 Class: WorkTask

Attribute	Description
Class Name	WorkTask
Parent Class	Task
Purpose	Represents work-related tasks with specific formatting. Extends Task and provides work-specific implementation of abstract methods.

Table 4: WorkTask Class Overview

Key Fields: Inherits all fields from Task class. No additional fields.

Key Methods:

Method	Return Type	Description
getTaskType()	String	Returns "WorkTask"
getDetails()	String	Returns formatted work task details with box-style borders
truncate(String, int)	String	Helper method to truncate long strings

Table 5: WorkTask Class Methods

2.3.3 Class: PersonalTask

Attribute	Description
Class Name	PersonalTask
Parent Class	Task
Purpose	Represents personal tasks with specific formatting. Extends Task and provides personal-specific implementation of abstract methods.

Table 6: PersonalTask Class Overview

Key Fields: Inherits all fields from Task class. No additional fields.

Key Methods:

Method	Return Type	Description
getTaskType()	String	Returns "PersonalTask"
getDetails()	String	Returns formatted personal task details with double-line borders
truncate(String, int)	String	Helper method to truncate long strings

Table 7: PersonalTask Class Methods

2.3.4 Class: TaskManager

Attribute	Description
Class Name	TaskManager
Parent Class	None
Purpose	Main application class that handles user interaction, task storage, and file I/O operations. Provides the menu-driven interface.

Table 8: TaskManager Class Overview

Key Fields (Encapsulated Variables):

Field	Type	Access	Description
tasks	ArrayList<Task>	private	Collection storing all tasks (polymorphic)
scanner	Scanner	private	Input reader for user interaction
DATA_FILE	static final String	private	File path for data persistence

Table 9: TaskManager Class Fields

Key Methods:

Method	Return Type	Description
main(String[])	static void	Application entry point
run()	void	Main application loop
displayMenu()	void	Shows menu options
addNewTask()	void	Creates new tasks
markTaskCompleted()	void	Marks tasks as done
displayAllTasks()	void	Shows all tasks
displayCompletedTasks()	void	Shows completed tasks
displayPendingTasks()	void	Shows pending tasks
saveTasksToFile()	void	Persists tasks to file
loadTasksFromFile()	void	Loads tasks from file
validateTaskIndex(int)	void	Validates task index, throws exception
parseTaskFromLine(String)	Task	Parses task from file line

Table 10: TaskManager Class Methods

2.3.5 Class: TaskNotFoundException

Attribute	Description
Class Name	TaskNotFoundException
Parent Class	Exception
Purpose	Custom exception class for handling invalid task access operations. Thrown when a user attempts to access a non-existent task.

Table 11: TaskNotFoundException Class Overview

Key Methods (Constructors):

Constructor	Description
TaskNotFoundException(String)	Creates exception with custom message
TaskNotFoundException(String, Throwable)	Creates exception with message and cause
TaskNotFoundException(int, int)	Creates exception with index range information

Table 12: TaskNotFoundException Constructors

2.4 Inheritance and Polymorphism

2.4.1 Inheritance Hierarchy

```
Exception
    +-- TaskNotFoundException

Task (abstract)
    +-- WorkTask
    +-- PersonalTask
```

2.4.2 Overridden Methods and Behavior Differences

Method	WorkTask Behavior	PersonalTask Behavior
getTaskType()	Returns "WorkTask"	Returns "PersonalTask"
getDetails()	Returns task details with single-line box borders	Returns task details with double-line box borders

Table 13: Overridden Methods Comparison

Polymorphism in Action: The `ArrayList<Task>` in TaskManager stores both WorkTask and PersonalTask objects. When iterating and calling `task.getDetails()`, Java's dynamic dispatch ensures the correct subclass method is executed based on the actual object type.

2.5 Encapsulation

Example from Task class:

```java
// Private fields - cannot be accessed directly from outside
private int id;
private String title;
private String description;
private boolean isCompleted;

// Public getters - provide read access
public int getId() { return id; }
public String getTitle() { return title; }

// Public setters - provide write access with potential validation
public void setTitle(String title) { this.title = title; }
public void setCompleted(boolean completed) {
    this.isCompleted = completed;
}
```

Listing 1: Encapsulation in Task Class

Benefits of Encapsulation Used:

1. **Data Protection**: External code cannot directly modify internal state
2. **Validation Opportunity**: Setters can validate input before assignment
3. **Implementation Flexibility**: Internal representation can change without affecting external code
4. **ID Management**: The `id` field has no setter, ensuring IDs remain unique and immutable

2.6 Exception Handling Strategy

2.6.1 Types of Exceptions Handled

Exception Type	Where Handled	Purpose
NumberFormatException	run(), addNewTask(), markTaskCompleted()	Handles invalid numeric input from user
InputMismatchException	run()	Catches scanner input type mismatches
TaskNotFoundException	markTaskCompleted()	Custom exception for invalid task index access
IOException	saveTasksToFile(), loadTasksFromFile()	Handles file read/write errors
Exception (general)	parseTaskFromLine()	Catches any parsing errors when loading file

Table 14: Exception Types and Handling Locations

2.6.2 Why Exception Handling is Necessary

1. **User Input Validation**: Users may enter text when numbers are expected
2. **Boundary Checking**: Users may request tasks with invalid indices
3. **File Operations**: File may be missing, corrupted, or inaccessible
4. **Program Robustness**: Prevents crashes and provides meaningful error messages

3 Implementation Details

3.1 Important Code Snippets

3.1.1 Constructor Example (Task.java)

```
public Task(String title, String description) {
    this.id = idCounter++;           // Auto-generate unique ID
    this.title = title;
    this.description = description;
    this.isCompleted = false;        // New tasks start as
       incomplete
}
```

Listing 2: Task Constructor

This constructor demonstrates encapsulation by initializing private fields and using a static counter for automatic ID generation.

3.1.2 Overridden Method Example (WorkTask.java)

```
@Override
public String getDetails() {
    String status = isCompleted() ? "Completed" : "Pending";
    return String.format(
        "[WORK TASK] ID: %d\n" +
        "Title: %s\n" +
        "Description: %s\n" +
        "Status: %s",
        getId(), truncate(getTitle(), 32),
        truncate(getDescription(), 27), status
    );
}
```

Listing 3: WorkTask getDetails() Method

This method overrides the abstract `getDetails()` from Task class, demonstrating polymorphism. Each subclass provides its own unique formatting.

3.1.3 Try-Catch Block Example (TaskManager.java)

```
try {
    String indexInput = scanner.nextLine().trim();
    int index = Integer.parseInt(indexInput);

    // Validate index and throw custom exception if invalid
    validateTaskIndex(index);

    Task task = tasks.get(index);
    task.setCompleted(true);
    System.out.println("\nTask marked as completed!");

} catch (NumberFormatException e) {
    System.out.println("\nInvalid input! Please enter a valid
       number.");
} catch (TaskNotFoundException e) {
    System.out.println("\nError: " + e.getMessage());
}
```

Listing 4: Exception Handling Example

This demonstrates exception handling with multiple catch blocks for different exception types.

3.1.4 Custom Exception Class (TaskNotFoundException.java)

```
public class TaskNotFoundException extends Exception {
    public TaskNotFoundException(String message) {
        super(message);
    }

    public TaskNotFoundException(int index, int maxIndex) {
        super("Task not found at index " + index +
              ". Valid range: 0 to " + maxIndex);
    }
}
```

Listing 5: Custom Exception Class

Custom exception providing meaningful error messages for invalid task access attempts.

3.2 User Interaction Flow

The program uses a menu-driven console interface:

Main Menu Display:

```
+---------------------------------------------+
|              MAIN MENU                      |
+---------------------------------------------+
|  1. Add a New Task                          |
|  2. Mark Task as Completed                  |
|  3. Display All Tasks                       |
|  4. Display Completed Tasks                 |
|  5. Display Pending Tasks                   |
|  6. Save Tasks to File                      |
|  7. Exit                                    |
+---------------------------------------------+
Enter your choice: _
```

Adding a Task Flow:

```
======= ADD NEW TASK =======

Enter task title: Complete Java Assignment
Enter task description: Finish the OOP project

Select task type:
  1. Work Task
  2. Personal Task
Enter choice (1 or 2): 1

Work Task added successfully!
  Task ID: 1
  Title: Complete Java Assignment
```

4 Testing and Results

4.1 Test Strategy

Feature	Test Method	Input Validation
Add Task	Manual testing with various inputs	Empty title check, invalid type selection
Mark Completed	Test with valid and invalid indices	Index bounds checking, already-completed check
Display Tasks	Visual inspection of formatting	Empty list handling
Save/Load	File creation and data integrity	File I/O error handling
Menu Navigation	All menu options tested	Invalid choice handling

Table 15: Test Strategy Overview

Input Validation Tests Performed:

- Empty task title → Shows error message, task not created
- Non-numeric menu choice → Caught by NumberFormatException
- Invalid task index → Throws TaskNotFoundException with helpful message
- Invalid task type selection → Shows error, returns to menu

4.2 Sample Outputs

4.2.1 Adding Tasks

```
======= ADD NEW TASK =======

Enter task title: Study for Exams
Enter task description: Review chapters 1-5

Select task type:
  1. Work Task
  2. Personal Task
Enter choice (1 or 2): 2

Personal Task added successfully!
  Task ID: 1
  Title: Study for Exams
```

4.2.2 Viewing Tasks

```
======= ALL TASKS =======

Total tasks: 2

[WORK TASK] ID: 1
Title: Complete Java Assignment
Description: Finish the OOP project
Status: Pending

[PERSONAL TASK] ID: 2
Title: Study for Exams
Description: Review chapters 1-5
Status: Completed
```

4.2.3 Marking Tasks as Completed

```
======= MARK TASK COMPLETED =======

Available Tasks:
----------------------------------------
  0. [ ] [Work] Complete Java Assignment
  1. [X] [Personal] Study for Exams
----------------------------------------

Enter the index of the task to mark as completed: 0

Task marked as completed!
  Title: Complete Java Assignment
```

4.2.4 Handling Errors

```
Enter your choice: abc
Invalid input! Please enter a valid number.

Enter the index of the task to mark as completed: 99
Error: Task not found at index 99. Valid range: 0 to 1
```

5 Conclusion

5.1 Project Achievements

The Smart Task Manager successfully implements a functional task management application that allows users to create, track, and manage tasks through an intuitive console interface. The application demonstrates proper use of Object-Oriented Programming principles including encapsulation, inheritance, polymorphism, and exception handling.

5.2 OOP Concepts Learned

Through this project, the following OOP concepts were practically applied:

- **Encapsulation**: Using private fields with public accessors protects data integrity
- **Inheritance**: Creating specialized task types (WorkTask, PersonalTask) from a common base class promotes code reuse
- **Polymorphism**: Storing different task types in a single collection and calling overridden methods enables flexible, extensible code
- **Abstraction**: The abstract Task class defines a contract that subclasses must fulfill
- **Exception Handling**: Custom exceptions and try-catch blocks make the application robust and user-friendly

5.3 Future Enhancements

Potential improvements for future versions:

1. **Priority Levels**: Add high/medium/low priority to tasks
2. **Due Dates**: Add deadline tracking with reminder functionality
3. **Task Categories**: Allow custom categories beyond Work/Personal
4. **Search Functionality**: Search tasks by keyword or filter by various criteria
5. **GUI Interface**: Develop a graphical user interface using JavaFX or Swing
6. **Database Storage**: Replace file storage with SQLite or MySQL database
7. **Task Editing**: Allow modification of existing task titles and descriptions
8. **Task Deletion**: Add ability to remove tasks from the list
9. **Statistics Dashboard**: Show completion rates and productivity metrics
10. **Export Options**: Export tasks to PDF or CSV format

6 References

1. CSE110: Object Oriented Programming - Lecture Slides (Fall 2025)
2. Oracle Java Documentation (Javadoc) - `https://docs.oracle.com/javase/`
3. Claude Opus (Anthropic AI Assistant) - Used for Terminal UI and debugging
4. Overleaf - Online LaTeX Editor for report writing.

End of the report